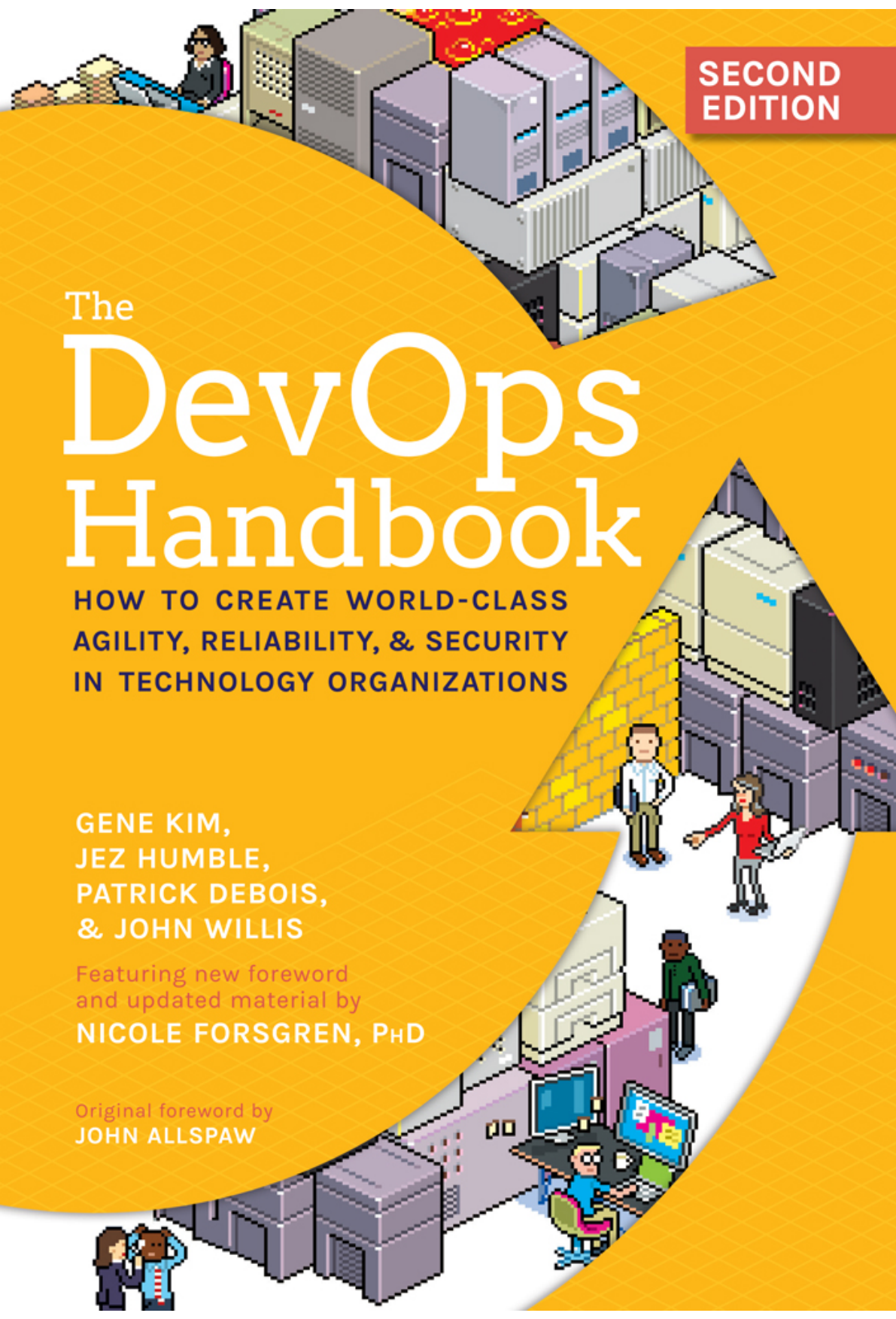




SECOND
EDITION

The DevOps Handbook

HOW TO CREATE WORLD-CLASS
AGILITY, RELIABILITY, & SECURITY
IN TECHNOLOGY ORGANIZATIONS

GENE KIM,
JEZ HUMBLE,
PATRICK DEBOIS,
& JOHN WILLIS

Featuring new foreword
and updated material by
NICOLE FORSGREN, PhD

Original foreword by
JOHN ALLSPAW

**SECOND
EDITION**

The
**DevOps
Handbook**

HOW TO CREATE WORLD-CLASS
AGILITY, RELIABILITY, & SECURITY
IN TECHNOLOGY ORGANIZATIONS

**SECOND
EDITION**

The
**DevOps
Handbook**

**HOW TO CREATE WORLD-CLASS
AGILITY, RELIABILITY, & SECURITY
IN TECHNOLOGY ORGANIZATIONS**

**GENE KIM, JEZ HUMBLE, PATRICK DEBOIS,
& JOHN WILLIS**

**FEATURING NEW FOREWORD AND UPDATED
MATERIALS FROM NICOLE FORSGREN, PhD**

ORIGINAL FOREWORD BY JOHN ALLSPAW

IT Revolution
Portland, Oregon



25 NW 23rd Pl, Suite 6314
Portland, OR 97210

All rights reserved. For information about permission to reproduce selections from
this book,
write to Permissions, IT Revolution Press, LLC,
25 NW 23rd Pl, Suite 6314, Portland, OR 97210

The DevOps Handbook, Second Edition © 2021 by Gene Kim, Matthew "Jez"
Humble,
Patrick Debois, John Willis, and Nicole Forsgren

First edition © 2016 by Gene Kim, Jez Humble, Patrick Debois, and John Willis
Printed in the United States of America

27 26 25 24 23 22 21 1 2 3 4 5 6 7 8 9 10

Cover design by Devon Smith Creative
Cover illustration by eboy
Book design by Devon Smith Creative

ISBN: 9781950508402
eBook ISBN: 9781950508433
Web PDF ISBN: 9781942788867
Audio ISBN: 9781950508440

The author of the 18F case study has dedicated the work to the public domain by
waiving all of his or her rights to the work worldwide under copyright law, including
all related
and neighboring rights, to the extent allowed by law. You can copy, modify,
distribute, and
perform case study 18F, even for commercial purposes, all without asking
permission.

For information about special discounts for bulk purchases
or for information on booking authors for an event,
please visit ITRevolution.com.

THE DEVOPS HANDBOOK, SECOND EDITION

CONTENTS

Figures & Tables

Note from the Publisher on the Second Edition

Foreword to the Second Edition: Nicole Forsgren

Foreword to the First Edition: John Allspaw

Preface

Introduction

Part I—The Three Ways

Part I Introduction

01 Agile, Continuous Delivery, and the Three Ways

NEW Case Study: Approaching Cruising Altitude: American Airlines' DevOps Journey (Part 1) (2020)

02 The First Way: The Principles of Flow

NEW Case Study: Flow and Constraint Management in Healthcare (2021)

03 The Second Way: The Principles of Feedback

NEW Case Study: Pulling the Andon Cord at Excella (2018)

04 The Third Way: The Principles of Continual Learning and Experimentation

NEW Case Study: The Story of Bell Labs (1925)

Part 1 Conclusion

Part II—Where to Start

Part II Introduction

05 Selecting Which Value Stream to Start With

Case Study: Nordstrom's DevOps Transformation (2014–2015)

NEW Case Study: Kessel Run: The Brownfield Transformation of a Mid-Air Refueling System (2020)

NEW Case Study: Scaling DevOps Across the Business: American Airlines' DevOps Journey (Part 2) (2020)

NEW Case Study: Saving the Economy From Ruin (With a Hyperscale PaaS) at HMRC (2020)

06 Understanding the Work in Our Value Stream, Making it Visible, and Expanding it Across the Organization

Case Study: Nordstrom's Experience with Value Stream Mapping (2015)

Case Study: Operation InVersion at LinkedIn (2011)

07 How to Design Our Organization and Architecture with Conway's Law in Mind

Case Study: Conway's Law at Etsy (2015)

Case Study: API Enablement at Target (2015)

08 How to Get Great Outcomes by Integrating Operations into the Daily Work of Development

Case Study: Big Fish Games (2014)

NEW Case Study: Better Ways of Working at Nationwide Building Society (2020)

Part II Conclusion

Part III—The First Way: The Technical Practices of Flow

Part III Introduction

09 Create the Foundations of Our Deployment Pipeline

Case Study: Enterprise Data Warehouse (2009)

NEW Case Study: How a Hotel Company Ran \$30B of Revenue in Containers (2020)

- 10 Enable Fast and Reliable Automated Testing
 - Case Study: Google Web Server (2005)
- 11 Enable and Practice Continuous Integration
 - Case Study: HP LaserJet Firmware (2006)
 - Case Study: Continuous Integration of Bazaarvoice (2012)
- 12 Automate and Enable Low-Risk Releases
 - Case Study: Daily Deployments at CSG International (2013)
 - Case Study: Etsy—Self-Service Developer Deployment: An Example of Continuous Deployment (2014)
 - Case Study: Dixons Retail—Blue-Green Deployment for Point-of-Sale System (2008)
 - Case Study: Dark Launch of Facebook Chat (2008)
 - NEW Case Study: Creating a Win-Win for Dev & Ops at CSG (2016)
- 13 Architect for Low-Risk Releases
 - Case Study: Evolutionary Architecture at Amazon (2002)
 - Case Study: Strangler Fig Pattern at Blackboard Learn (2011)
- Part III Conclusion

Part IV—The Second Way: The Technical Practices of Feedback

Part IV Introduction

- 14 Create Telemetry to Enable Seeing and Solving Problems
 - Case Study: DevOps Transformation at Etsy (2012)
 - Case Study: Creating Self-Service Metrics at LinkedIn (2011)
- 15 Analyze Telemetry to Better Anticipate Problems and Achieve Goals
 - Case Study: Telemetry at Netflix (2012)
 - Case Study: Auto-Scaling Capacity at Netflix (2012)
 - Case Study: Advanced Anomaly Detection (2014)

16 Enable Feedback So Development and Operations Can Safely Deploy Code

Case Study: Right Media (2006)

Case Study: The Launch and HandOff Readiness Review
Google (2010)

17 Integrate Hypothesis-Driven Development and A/B Testing into Our Daily Work

Case Study: Hypothesis-Driven Development at Intuit, Inc.
(2012)

Case Study: Doubling Revenue Growth through Fast Release
Cycle Experimentation at Yahoo! Answers (2010)

18 Create Review and Coordination Processes to Increase Quality of Our Current Work

Case Study: Peer Review at GitHub (2011)

NEW Case Study: From Six-Eye Principle to Release at Scale
at Adidas (2020)

Case Study: Code Reviews at Google (2010)

Case Study: Pair Programming Replacing Broken Code Review
Processes at Pivotal Labs (2011)

Part IV Conclusion

Part V—The Third Way: The Technical Practices of Continual Learning and Experimentation

Part V Introduction

19 Enable and Inject Learning into Daily Work

Case Study: AWS US-East and Netflix (2011)

NEW Case Study: Turning an Outage into a Powerful
Learning Opportunity at CSG (2020)

20 Convert Local Discoveries into Global Improvements

Case Study: Standardizing a New Technology Stack at Etsy (2010)

NEW Case Study: Crowdsourcing Technology Governance at Target (2018)

21 Reserve Time to Create Organizational Learning and Improvement

Case Study: Thirty-Day Challenge at Target (2015)

Case Study: Internal Technology Conferences at Nationwide Insurance, Capital One, and Target (2014)

Part V Conclusion

Part VI—The Technological Practices of Integrating Information Security, Change Management, and Compliance

Part VI Introduction

22 Information Security Is Everyone's Job Every Day

Case Study: Static Security Testing at Twitter (2009)

Case Study: 18F Automating Compliance for the Federal Government with Compliance Masonry (2016)

Case Study: Instrumenting the Environment at Etsy (2010)

NEW Case Study: Shifting Security Left at Fannie Mae (2020)

23 Protecting the Deployment Pipeline

Case Study: Automated Infrastructure Changes as Standard Changes at Salesforce.com (2012)

Case Study: PCI Compliance and a Cautionary Tale of Separating Duties at Etsy (2014)

NEW Case Study: Biz and Tech Partnership toward Ten "No Fear Releases" Per Day at Capital One (2020)

Case Study: Proving Compliance in Regulated Environments (2015)

Case Study: Relying on Production Telemetry for ATM
Systems (2013)

Part VI Conclusion

A Call to Action: Conclusion to *The DevOps Handbook*

Afterword to the Second Edition

Appendices

Bibliography

Notes

Index

Acknowledgments

About the Authors

FIGURES & TABLES

Table 0.1: **The Ever Accelerating Trend toward Faster, Cheaper, Lower Risk Delivery of Software**

Figure 0.1: **Deployments per Day vs. Number of Developers**

Figure 1.1: **Lead Time vs. Process Time of a Deployment Operation**

Figure 1.2: **A Technology Value Stream with a Deployment Lead Time of Three Months**

Figure 1.3: **A Technology Value Stream with a Lead Time of Minutes**

Figure 1.4: **The Three Ways**

Figure 1.5: **American Airlines' DevOps Transformation Journey**

Figure 2.1: **An Example Kanban Board Spanning Requirements, Dev, Test, Staging, and In Production**

Figure 2.2: **Simulation of "Envelope Game"**

Figure 3.1: **Feedback Cycle Times**

Figure 3.2: **Cycle Time vs. Andon Pulls at Excella**

Table 4.1: **The Westrum Organizational Typology Model**

Figure 5.1: **The Technology Adoption Curve**

Figure 5.2: **American Airlines' Delivery Transformation**

Table 5.1: **American Airlines' New Vocabulary**

Figure 6.1: **An Example Value Stream Map**

Figure 6.2: **Invest 20% of Capacity in those Who Create Positive, User-Invisible Value**

Figure 7.1: **Functional vs. Market Orientation**

Table 7.1: **Specialists vs. Generalists vs. "E-shaped" Staff**

Figure 8.1: **Functional Teams in Silos vs. Long-Lived, Multi-Skilled**

Teams

Figure 10.1: **The Deployment Pipeline**

Figure 10.2: **The Ideal and Non-Ideal Automated Testing Pyramids**

Figure 10.3: **Running Automated and Manual Tests in Parallel**

Figure 12.1: **Number of Developers Deploying per Week at Facebook**

Figure 12.2: **Daily Deployments at CSG International**

Figure 12.3: **Elite and High Performers Achieve Faster Deployment Lead Times and MTTR (2019)**

Figure 12.4: **The Deployinator Console at Etsy**

Figure 12.5: **Blue-Green Deployment Patterns**

Figure 12.6: **The Canary Release Pattern**

Figure 12.7: **How Structure Influences Behavior and Quality**

Figure 12.8: **From Siloed Approach to Cross-Functional Teams**

Figure 12.9: **Conventional vs. Cross-Functional Structure**

Figure 13.1: **Google Cloud Datastore**

Table 13.1: **Architectural Archetypes**

Figure 13.2: **Blackboard Learn Code Repository: Before Building Blocks**

Figure 13.3: **Blackboard Learn Code Repository: After Building Blocks**

Figure 14.1: **Incident Resolution Time for Elite, High, Medium, and Low Performers (2019)**

Figure 14.2: **Monitoring Framework**

Figure 14.3: **One Line of Code to Generate Telemetry using StatsD and Graphite at Etsy**

Figure 14.4: **User Excitement of New Features in User Forum Posts after Deployments**

Figure 15.1: **Standard Deviations (σ)Mean (μ) with Gaussian**

Distribution

- Figure 15.2: Downloads per Minute: Over-Alerting when Using “Three Standard Deviation” Rule
- Figure 15.3: Downloads per Minute: Histogram of Data Showing Non-Gaussian Distribution
- Figure 15.4: Netflix Customer Viewing Demand for Five Days
- Figure 15.5: Netflix Stryer Forecasting Customer Traffic and the Resulting AWS Schedule of Computer Resources
- Figure 15.6: Autodesk Share Price and Thirty-Day Moving Average Filter
- Figure 15.7: Transaction Volume: Under-Alerting Using “Three Standard Deviation” Rule
- Figure 15.8: Transaction Volume: Using Kolmogorov-Smirnov Test to Alert on Anomalies
- Figure 16.1: Deployment to Etsy.com Causes PHP Run-Time Warnings and Is Quickly Fixed
- Figure 16.2: The “Service Handback” at Google
- Figure 16.3: The LRR and HRR at Google
- Figure 18.1: Comments and Suggestions on a GitHub Pull Request
- Figure 18.2: Size of Change vs. Lead Time for Reviews at Google
- Figure 21.1: The ASREDS Learning Loop
- Figure 22.1: Jenkins Running Automated Security Testing
- Figure 22.2: Number of Brakeman Security Vulnerabilities Detected
- Figure 22.3: Time to Remediate vs. Time to Update Dependencies (TTU)
- Figure 22.4: Five Behavioral Clusters for Open-Source Projects
- Figure 22.5: Developers See SQL Injection Attempts in Graphite at Etsy
- Figure AF.1: Average Development Window by Day of Week per User

Figure A.1: **The Core, Chronic Conflict Facing Every IT Organization**

Table A.1: **The Downward Spiral**

Figure A.2: **Queue Size and Wait Times as Function of Percent Utilization**

Table A.2: **Two Stories**

Figure A.3: **The Toyota Andon Cord**

NOTE FROM THE PUBLISHER ON THE SECOND EDITION

Impact of the First Edition

Since the original publication of *The DevOps Handbook*, data from the *State of DevOps Reports* and other research continue to show that DevOps improves time to value for businesses and increases productivity and worker well-being. It also helps create nimble, agile businesses that can adjust to overwhelming change, as witnessed in the COVID-19 pandemic of 2020 and beyond.

“I think 2020 has been illuminating in showing what technology can do in a time of incredible crisis,” Gene Kim said in his “State of DevOps: 2020 and Beyond” article. “The crisis provided a catalyst for rapid change. And I’m thankful we were able to rise and meet it.”¹

One of the underpinnings of DevOps and *The DevOps Handbook* is that it shows—and is indeed written for—the horses not the unicorns of the business and technology world. DevOps was never, and still is not, only effective at technology giants—the FAANGs—or startups. This book and the DevOps community as a whole have shown time and time again that DevOps practices and processes can take even the most legacy-riddled, old “horse” enterprise organization and turn it into a nimble technology organization.

In 2021, it is clearer than ever before that every business is a technology business and every leader is a technology leader. Not only can technology no longer be ignored or relegated to the basements; it must also be considered a vital part of the entire strategic endeavor of the business.

Changes to the Second Edition

In this expanded edition of *The DevOps Handbook*, the authors have updated the main text where new research, learnings, and experiences have developed and shaped our understanding of DevOps and how it is used in the industry. Additionally we are pleased to include renowned researcher Dr. Nicole Forsgren as co-author to help update and expand the text with new research and supporting metrics.

CONTINUOUS LEARNING

We've added some additional insights and resources we've learned since the first edition came out. These "Continuous Learning" sections are highlighted throughout the book as you see here and include new supporting data and additional resources, tools, and techniques to use on your DevOps journey.

We've also expanded the book with additional case studies to illustrate how far DevOps has spread throughout all industries, especially how it has spread beyond the IT department and into the C-suite itself. In addition, at the end of each case study we have added a key takeaway or two that highlight the most important, though not exclusive, lessons learned. Finally, we've updated the conclusion to each part with new resources to continue your learning journey.

What's Next for DevOps and the Age of Software

If the past five years have taught us anything, it is how important technology is and how much we can achieve when IT and the business speak openly and honestly, as DevOps facilitates.

Perhaps nothing illustrates this more than the rapid changes that were necessary due to the COVID-19 pandemic of 2020 and beyond. Through the use of DevOps, organizations mobilized technology to get services to customers, internal and external, in a moment of rapid, unprecedented change. These large, complex organizations, known for their inability to pivot and adapt quickly, suddenly had no other choice.

American Airlines also was able to take advantage of their ongoing DevOps transformation to build big wins quickly, as you can read about in [Chapters 1 and 5](#).

Dr. Chris Strear relates his experiences using the Theory of Constraints to optimize flow in hospitals, as you can read about in [Chapter 2](#).

In 2020 Nationwide Building Society, the world's largest mutual financial institution, was able to respond in weeks to customer needs versus the typical years, thanks to their ongoing DevOps transformation. You can read more about their experience in [Chapter 8](#).

But while technology is a piece of a successful transformation into future ways of working, business leadership must lead the charge. The bottleneck of today is no longer just technical practices (though they still exist); the biggest challenge and necessity is getting business leadership on board. Transformation must be co-created between the business and technology, and the theories presented here can lead that change.

The enterprise can no longer sustain a binary thought process: top down or tech only. We must achieve true collaboration. Ninety percent of that work involves getting the right people engaged, onboard, and aligned. Start there and we can maintain the resulting motivation into the future.

—IT Revolution
Portland, OR
June 2021

FOREWORD TO THE SECOND EDITION

It has been five years since the first edition of *The DevOps Handbook* was released. While so much has changed, so much has also remained the same. Some of our tools and technologies are no longer in vogue or might not exist, but that shouldn't detract any readers. Although the technology landscape has shifted, the underlying principles presented in this book remain as relevant as ever.

In fact, the need for DevOps is even greater today, as organizations need to deliver value quickly, safely, securely, and reliably to their customers and users. To do this, they need to transform their internal processes and leverage technology to deliver value—using DevOps practices. This is true for organizations around the world and across all industry verticals.

Over the past several years, I've led research in the annual *State of DevOps Reports* (first with Puppet and later with DORA and Google), with co-authors Jez Humble and Gene Kim. The research has confirmed that many of the practices described in this book lead to improved outcomes like speed and stability of software releases; reductions in deployment pain and burnout of our engineers; and contributions to organizational performance, including profitability, productivity, customer satisfaction, effectiveness, and efficiency.

For the second edition of *The DevOps Handbook*, we have refreshed the text with updated data based on the latest research and best practices, and

included new case studies to share even more stories about what transformations look like “on the ground.” Thanks for joining us on this journey of continuous improvement.

—Nicole Forsgren, PhD
Partner at Microsoft Research
2021

FOREWORD TO THE FIRST EDITION

In the past, many fields of engineering have experienced a sort of notable evolution, continually “leveling up” its understanding of its own work. While there are university curriculums and professional support organizations situated within specific disciplines of engineering (civil, mechanical, electrical, nuclear, etc.), the fact is, modern society needs all forms of engineering to recognize the benefits of and work in a multidisciplinary way.

Think about the design of a high-performance vehicle. Where does the work of a mechanical engineer end and the work of an electrical engineer begin? Where (and how, and when) should someone with domain knowledge of aerodynamics (who certainly would have well-formed opinions on the shape, size, and placement of windows) collaborate with an expert in passenger ergonomics? What about the chemical influences of fuel mixture and oil on the materials of the engine and transmission over the lifetime of the vehicle? There are other questions we can ask about the design of an automobile, but the end result is the same: success in modern technical endeavors absolutely requires multiple perspectives and expertise to collaborate.

In order for a field or discipline to progress and mature, it needs to reach a point where it can thoughtfully reflect on its origins, seek out a diverse set of perspectives on those reflections, and place that synthesis into a context that is useful for how the community pictures the future.

This book represents such a synthesis and should be seen as a seminal collection of perspectives on the (I will argue, still emerging and quickly evolving) field of software engineering and operations.

No matter what industry you are in, or what product or service your organization provides, this way of thinking is paramount and necessary for survival for every business and technology leader.

—John Allspaw, CTO, Etsy
Brooklyn, NY, August 2016

PREFACE

Aha!

The journey to complete *The DevOps Handbook* has been a long one—it started with weekly working Skype calls between the co-authors in February of 2011, with the vision of creating a prescriptive guide that would serve as a companion to the as-yet-unfinished book *The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win*.

More than five years later, with over two thousand hours of work, *The DevOps Handbook* is finally here. Completing this book has been an extremely long process, although one that has been highly rewarding and full of incredible learning, with a scope that is much broader than we originally envisioned. Throughout the project, all the co-authors shared a belief that DevOps is genuinely important, formed in a personal “aha” moment much earlier in each of our professional careers, which I suspect many of our readers will resonate with.

Gene Kim

I’ve had the privilege of studying high-performing technology organizations since 1999, and one of the earliest findings was that boundary-spanning between the different functional groups of IT Operations, Information Security, and Development was critical to success. But I still remember the first time I saw the magnitude of the

downward spiral that would result when these functions worked toward opposing goals.

It was 2006, and I had the opportunity to spend a week with the group that managed the outsourced IT Operations of a large airline reservation service. They described the downstream consequences of their large, annual software releases: each release would cause immense chaos and disruption for the outsourcer as well as customers; there would be SLA (service level agreement) penalties because of the customer-impacting outages; there would be layoffs of the most talented and experienced staff because of the resulting profit shortfalls; there would be much unplanned work and firefighting so that the remaining staff couldn't work on the ever-growing service request backlogs coming from customers; the contract would be held together by the heroics of middle management; and everyone felt that the contract would be doomed to be put out for re-bid in three years.

The sense of hopelessness and futility that resulted created, for me, the beginnings of a moral crusade. Development seemed to always be viewed as strategic, but IT Operations was viewed as tactical, often delegated away or outsourced entirely, only to return in five years in worse shape than it was first handed over.

For many years, many of us knew that there must be a better way. I remember seeing the talks coming out of the 2009 Velocity Conference, describing amazing outcomes enabled by architecture, technical practices, and cultural norms that we now know as DevOps. I was so excited because it clearly pointed to the better way that we had all been searching for. And helping spread that word was one of my personal motivations to co-author *The Phoenix Project*. You can imagine how incredibly rewarding it was to see the broader community react to that book, describing how it helped them achieve their own “aha” moments.

Jez Humble

My DevOps “aha” moment was at a startup in 2000—my first job after graduating. For some time, I was one of two technical staff. I did everything: networking, programming, support, systems administration. We deployed software to production by FTP directly from our work-stations.

Then in 2004 I got a job at ThoughtWorks, a consultancy where my first gig was working on a project involving about seventy people. I was on a team of eight engineers whose full-time job was to deploy our software into a production-like environment. In the beginning, it was really stressful. But over a few months, we went from manual deployments that took two weeks to an automated deployment that took one hour, where we could roll forward and back in milliseconds using the blue-green deployment pattern during normal business hours.

That project inspired a lot of the ideas in both the *Continuous Delivery* book and this one. A lot of what drives me and others working in this space is the knowledge that, whatever your constraints, we can always do better, and the desire to help people on their journey.

Patrick Debois

For me, it was a collection of moments. In 2007 I was working on a data center migration project with some Agile teams. I was jealous that they had such high productivity—able to get so much done in so little time.

For my next assignment, I started experimenting with Kanban in Operations and saw how the dynamic of the team changed. Later, at the Agile Toronto 2008 conference, I presented my IEEE paper on this, but I felt it didn’t resonate widely in the Agile community. We

started an Agile system administration group, but I overlooked the human side of things.

After seeing the 2009 Velocity Conference presentation “10 Deploys per Day” by John Allspaw and Paul Hammond, I was convinced others were thinking in a similar way. So I decided to organize the first DevOpsDays, accidentally coining the term DevOps.

The energy at the event was unique and contagious. When people started to thank me because it changed their life for the better, I understood the impact. I haven’t stopped promoting DevOps since.

John Willis

In 2008, I had just sold a consulting business that focused on large-scale, legacy IT operations practices around configuration management and monitoring (Tivoli) when I first met Luke Kanies (the founder of Puppet Labs). Luke was giving a presentation on Puppet at an O’Reilly open-source conference on configuration management (CM).

At first I was just hanging out at the back of the room, killing time and thinking, “What could this twenty-year-old tell me about configuration management?” After all, I had literally been working my entire life at some of the largest enterprises in the world, helping them architect CM and other operations management solutions. However, about five minutes into his session, I moved up to the first row and realized everything I had been doing for the last twenty years was wrong. Luke was describing what I now call second-generation CM.

After his session, I had an opportunity to sit down and have coffee with him. I was totally sold on what we now call infrastructure as code. However, while we met for coffee, Luke started going even further, explaining his ideas. He started telling me he believed that

Operations was going to have to start behaving like software developers. They were going to have to keep their configurations in source control and adopt CI/CD delivery patterns for their workflow. Being the old IT Operations person at the time, I think I replied to him with something like, “That idea is going to sink like Led Zeppelin with Ops folk.” (I was clearly wrong.)

Then about a year later, in 2009, at another O’Reilly conference, Velocity, I saw Andrew Clay Shafer give a presentation on Agile infrastructure. In his presentation, Andrew showed this iconic picture of a wall between developers and Operations with a metaphorical depiction of work being thrown over the wall. He coined this “the wall of confusion.” The ideas he expressed in that presentation codified what Luke was trying to tell me a year earlier. That was the light bulb for me. Later that year, I was the only American invited to the original DevOpsDays in Ghent. By the time that event was over, this thing we call DevOps was clearly in my blood.

DevOps Myths

Clearly, the co-authors of this book all came to a similar epiphany, even if they came there from very different directions. But there is now an overwhelming weight of evidence that the problems described above happen almost everywhere, and that the solutions associated with DevOps are nearly universally applicable.

The goal of writing this book is to describe how to replicate the DevOps transformations we’ve been a part of or have observed, as well as dispel many of the myths of why DevOps won’t work in certain situations. Below are some of the most common myths we hear about DevOps.

Myth—DevOps Is Only for Startups: While DevOps practices have been pioneered by the web-scale, internet “unicorn” companies such as Google, Amazon, Netflix, and Etsy, each of these organizations has, at some point

in their history, risked going out of business because of the problems associated with more traditional “horse” organizations: highly dangerous code releases that were prone to catastrophic failure, inability to release features fast enough to beat the competition, compliance concerns, an inability to scale, high levels of distrust between Development and Operations, and so forth.

However, each of these organizations was able to transform their architecture, technical practices, and culture to create the amazing outcomes that we associate with DevOps. As Dr. Branden Williams, an information security executive, said, “Let there be no more talk of DevOps unicorns or horses but only thoroughbreds and horses heading to the glue factory.”¹

Myth—DevOps Replaces Agile: DevOps principles and practices are compatible with Agile, with many observing that DevOps is a logical continuation of the Agile journey that started in 2001. Agile often serves as an effective enabler of DevOps because of its focus on small teams continually delivering high-quality code to customers.

Many DevOps practices emerge if we continue to manage our work beyond the goal of “potentially shippable code” at the end of each iteration, extending it to having our code always in a deployable state, with developers checking into trunk daily, and if we demonstrate our features in production-like environments.

Myth—DevOps Is Incompatible with ITIL: Many view DevOps as a backlash to ITIL or ITSM (IT Service Management), which was originally published in 1989. ITIL has broadly influenced multiple generations of Ops practitioners, including one of the co-authors, and is an ever-evolving library of practices intended to codify the processes and practices that underpin world-class IT Operations, spanning service strategy, design, and support.

DevOps practices can be made compatible with ITIL processes. To support the shorter lead times and higher deployment frequencies associated with DevOps, many areas of ITIL become fully automated, solving many problems associated with the configuration and release management processes (e.g., keeping the configuration management database and definitive software libraries up to date). And because DevOps requires fast detection and recovery when service incidents occur, the ITIL disciplines of service design, incident, and problem management remain as relevant as ever.

Myth—DevOps Is Incompatible with Information Security and Compliance: The absence of traditional controls (e.g., segregation of duty, change approval processes, manual security reviews at the end of the project) may dismay information security and compliance professionals.

However, that doesn't mean that DevOps organizations don't have effective controls. Instead of security and compliance activities only being performed at the end of the project, controls are integrated into every stage of daily work in the software development life cycle, resulting in better quality, security, and compliance outcomes.

Myth—DevOps Means Eliminating IT Operations, or “NoOps”: Many misinterpret DevOps as the complete elimination of the IT Operations function. However, this is rarely the case. While the nature of IT Operations work may change, it remains as important as ever. IT Operations collaborates far earlier in the software life cycle with Development, who continues to work with IT Operations long after the code has been deployed into production.

Instead of IT Operations doing manual work that comes from work tickets, it enables developer productivity through APIs and self-serviced platforms that create environments, test and deploy code, monitor and display production telemetry, and so forth. By doing this, IT Operations becomes more like Development (as do QA and Infosec), engaged in

product development, where the product is the platform that developers use to safely, quickly, and securely test, deploy, and run their IT services in production.

Myth—DevOps Is Just “Infrastructure as Code” or Automation: While many of the DevOps patterns shown in this book require automation, DevOps also requires cultural norms and an architecture that allows for the shared goals to be achieved throughout the IT value stream. This goes far beyond just automation. As Christopher Little, a technology executive and one of the earliest chroniclers of DevOps, wrote, “DevOps isn’t about automation, just as astronomy isn’t about telescopes.”²

Myth—DevOps Is Only for Open-Source Software: Although many DevOps success stories take place in organizations using software such as the LAMP stack (Linux, Apache, MySQL, PHP), achieving DevOps outcomes is independent of the technology being used. Successes have been achieved with applications written in [Microsoft.NET](#), COBOL, and mainframe assembly code, as well as with SAP and even embedded systems (e.g., HP LaserJet firmware).

Spreading the Aha! Moment

Each of the authors has been inspired by the amazing innovations happening in the DevOps community and the outcomes they are creating, including safe systems of work and enabling small teams to quickly and independently develop and validate code that can be safely deployed to customers. Given our belief that DevOps is a manifestation of creating dynamic, learning organizations that continually reinforce high trust cultural norms, it is inevitable that these organizations will continue to innovate and win in the marketplace.

It is our sincere hope that *The DevOps Handbook* will serve as a valuable resource for many people in different ways:

- a guide for planning and executing DevOps transformations
- a set of case studies to research and learn from
- a chronicle of the history of DevOps
- a means to create a coalition that spans Product Owners, Architecture, Development, QA, IT Operations, and Information Security to achieve common goals
- a way to get the highest levels of leadership support for DevOps initiatives, as well as a moral imperative to change the way we manage technology organizations to enable better effectiveness and efficiency, as well as enabling a happier and more humane work environment, helping everyone become lifelong learners—this not only helps everyone achieve their highest goals as human beings, but also helps their organizations win

INTRODUCTION

Imagine a World Where Dev and Ops Become DevOps

Imagine a world where product owners, Development, QA, IT Operations, and Infosec work together, not only to help each other, but also to ensure that the overall organization succeeds. By working toward a common goal, they enable the fast flow of planned work into production (e.g., performing tens, hundreds, or even thousands of code deploys per day), while achieving world-class stability, reliability, availability, and security.

In this world, cross-functional teams rigorously test their hypotheses of which features will most delight users and advance the organizational goals. They care not just about implementing user features, but also about actively ensuring their work flows smoothly and frequently through the entire value stream without causing chaos and disruption to IT Operations or any other internal or external customer.

Simultaneously, QA, IT Operations, and Infosec are always working on ways to reduce friction for the team, creating the work systems that enable developers to be more productive and get better outcomes. By adding the expertise of QA, IT Operations, and Infosec into delivery teams and automated self-service tools and platforms, teams are able to use that expertise in their daily work without being dependent on other teams.

This enables organizations to create a safe system of work, where small teams are able to quickly and independently develop, test, and deploy code and value quickly, safely, securely, and reliably to customers. This allows organizations to maximize developer productivity, enable organizational learning, create high employee satisfaction, and win in the marketplace.

These are the outcomes that result from DevOps. For most of us, this is not the world we live in. More often than not, the system we work in is broken, resulting in extremely poor outcomes that fall well short of our true potential. In our world, Development and IT Operations are adversaries; testing and Infosec activities happen only at the end of a project, too late to correct any problems found; and almost any critical activity requires too much manual effort and too many handoffs, leaving us always waiting. Not only does this contribute to extremely long lead times to get anything done, but the quality of our work, especially production deployments, is also problematic and chaotic, resulting in negative impacts to our customers and our business.

As a result, we fall far short of our goals, and the whole organization is dissatisfied with the performance of IT, resulting in budget reductions and frustrated, unhappy employees who feel powerless to change the process and its outcomes.* The solution? We need to change how we work; DevOps shows us the best way forward.

To better understand the potential of the DevOps revolution, let us look at the manufacturing revolution of the 1980s. By adopting Lean principles and practices, manufacturing organizations dramatically improved plant productivity, customer lead times, product quality, and customer satisfaction, enabling them to win in the marketplace.

Before the revolution, average manufacturing plant order lead times were six weeks, with fewer than 70% of orders shipped on time. By 2005, with the widespread implementation of Lean practices, average product lead times had dropped to less than three weeks, and more than 95% of

orders were shipped on time.¹ Organizations that did not implement Lean practices lost market share, and many went out of business entirely.

Similarly, the bar has been raised for delivering technology products and services—what was good enough in previous decades is not good enough now. For each of the last four decades, the cost and time required to develop and deploy strategic business capabilities and features have dropped by orders of magnitude. During the 1970s and 1980s, most new features required one to five years to develop and deploy, often costing tens of millions of dollars.

By the 2000s, because of advances in technology and the adoption of Agile principles and practices, the time required to develop new functionality had dropped to weeks or months but deploying into production still required weeks or months, often with catastrophic outcomes.

And by 2010, with the introduction of DevOps and the never-ending commoditization of hardware, software, and now the cloud, features (even entire startup companies) could be created in weeks, quickly being deployed into production in just hours or minutes—for these organizations, deployment finally became routine and low risk. These organizations are able to perform experiments to test business ideas, discovering which ideas create the most value for customers and the organization as a whole, and which are then further developed into features that can be rapidly and safely deployed into production.

Table 0.1: The Ever-Accelerating Trend toward Faster, Cheaper, Lower Risk Delivery of Software

	1970s–1980s	1990s	2000s–Present
Era	Mainframes	Client/Server	Commoditization and Cloud
Representative technology of era	COBOL, DB2 on MVS, etc.	C++, Oracle, Solaris, etc.	Java, MySQL, Red Hat, Ruby on Rails, PHP, etc.
Cycle time	1–5 years	3–12 months	2–12 weeks
Cost	\$1M–\$100M	\$100k–\$10M	\$10k–\$1M
At risk	The whole company	A product line or division	A product feature
Cost of failure	Bankruptcy, sell the company, massive layoffs	Revenue miss, CIO's job	Negligible

Source: Adrian Cockcroft, “Velocity and Volume (or Speed Wins),” presentation at FlowCon, San Francisco, CA, November 2013.

Today, organizations adopting DevOps principles and practices often deploy changes hundreds or even thousands of times per day. In an age where competitive advantage requires fast time to market and relentless experimentation, organizations that are unable to replicate these outcomes are destined to lose in the marketplace to more nimble competitors and could potentially go out of business entirely, much like the manufacturing organizations that did not adopt Lean principles.

These days, regardless of what industry we are competing in, the way we acquire customers and deliver value to them is dependent on the technology value stream. Put even more succinctly, as Jeffrey Immelt, CEO of General Electric, stated, “Every industry and company that is not

bringing software to the core of their business will be disrupted.”² Or as Jeffrey Snover, Technical Fellow at Microsoft, said, “In previous economic eras, businesses created value by moving atoms. Now they create value by moving bits.”³

It’s difficult to overstate the enormity of this problem—it affects every organization, independent of the industry we operate in, the size of our organization, whether we are profit or nonprofit. Now more than ever, how technology work is managed and performed predicts whether our organizations will win in the marketplace or even survive. In many cases, we will need to adopt principles and practices that look very different from those that have successfully guided us over the past decades. (See [Appendix 1](#).)

Now that we have established the urgency of the problem that DevOps solves, let us take some time to explore in more detail the symptomatology of the problem, why it occurs, and why, without dramatic intervention, the problem worsens over time.

The Problem: Something in Your Organization Must Need Improvement (Or You Wouldn’t Be Reading This Book)

Most organizations are not able to deploy production changes in minutes or hours, instead requiring weeks or months. Nor are they able to deploy hundreds or thousands of changes into production per day; instead, they struggle to deploy monthly or even quarterly. Nor are production deployments routine, instead involving outages and chronic firefighting and heroics.

In an age where competitive advantage requires fast time to market, high service levels, and relentless experimentation, these organizations are at a significant competitive disadvantage. This is in large part due to their inability to resolve a core, chronic conflict within their technology organization.

The Core, Chronic Conflict

In almost every IT organization, there is an inherent conflict between Development and IT Operations that creates a downward spiral, resulting in ever-slower time to market for new products and features, reduced quality, increased outages, and, worst of all, an ever-increasing amount of technical debt.

The term “technical debt” was first coined by Ward Cunningham. Analogous to financial debt, technical debt describes how decisions we make lead to problems that get increasingly more difficult to fix over time, continually reducing our available options in the future—even when taken on judiciously, we still incur interest.

One factor that contributes to this is the often competing goals of Development and IT Operations. IT organizations are responsible for many things. Among them are the two following goals, which must be pursued simultaneously:

- Respond to the rapidly changing competitive landscape.
- Provide stable, reliable, and secure service to the customer.

Frequently, Development will take responsibility for responding to changes in the market and for deploying features and changes into production as quickly as possible. IT Operations will take responsibility for providing customers with IT service that is stable, reliable, and secure, making it difficult or even impossible for anyone to introduce production changes that could jeopardize production. Configured this way, Development and IT Operations have diametrically opposed goals and incentives.

Dr. Eliyahu M. Goldratt, one of the founders of the manufacturing management movement, called these types of configuration “the core, chronic conflict”—when organizational measurements and incentives

across different silos prevent the achievement of global, organizational goals.^{4†}

This conflict creates a downward spiral so powerful it prevents the achievement of desired business outcomes, both inside and outside the IT organization. These chronic conflicts often put technology workers into situations that lead to poor software and service quality and bad customer outcomes, as well as a daily need for workarounds, firefighting, and heroics, whether in Product Management, Development, QA, IT Operations, or Information Security. (See [Appendix 2](#).)

Downward Spiral in Three Acts

The downward spiral in IT has three acts that are likely familiar to most IT practitioners. The first act begins in IT Operations, where our goal is to keep applications and infrastructure running so that our organization can deliver value to customers. In our daily work, many of our problems are due to applications and infrastructure that are complex, poorly documented, and incredibly fragile. This is the technical debt and daily workarounds that we live with constantly, always promising that we'll fix the mess when we have a little more time. But that time never comes.

Alarmingly, our most fragile artifacts support either our most important revenue-generating systems or our most critical projects. In other words, the systems most prone to failure are also our most important and are at the epicenter of our most urgent changes. When these changes fail, they jeopardize our most important organizational promises, such as availability to customers, revenue goals, security of customer data, accurate financial reporting, and so forth.

The second act begins when somebody has to compensate for the latest broken promise—it could be a product manager promising a bigger, bolder feature to dazzle customers with or a business executive setting an even larger revenue target. Then, oblivious to what technology can or can't do,

or what factors led to missing our earlier commitment, they commit the technology organization to deliver upon this new promise.

As a result, Development is tasked with another urgent project that inevitably requires solving new technical challenges and cutting corners to meet the promised release date, further adding to our technical debt—made, of course, with the promise that we'll fix any resulting problems when we have a little more time.

This sets the stage for the third and final act, where everything becomes just a little more difficult, bit by bit—everybody gets a little busier, work takes a little more time, communications become a little slower, and work queues get a little longer. Our work becomes more tightly coupled, smaller actions cause bigger failures, and we become more fearful and less tolerant of making changes. Work requires more communication, coordination, and approvals; teams must wait just a little longer for their dependent work to get done; and our quality keeps getting worse. The wheels begin grinding slower and require more effort to keep turning. (See [Appendix 3](#).)

Although it's difficult to see in the moment, the downward spiral is obvious when one takes a step back. We notice that production code deployments are taking ever-longer to complete, moving from minutes to hours to days to weeks. And worse, the deployment outcomes have become even more problematic, resulting in an ever-increasing number of customer-impacting outages that require more heroics and firefighting in Operations, further depriving them of their ability to pay down technical debt.

As a result, our product delivery cycles continue to move slower and slower, fewer projects are undertaken, and those that are, are less ambitious. Furthermore, the feedback on everyone's work becomes slower and weaker, especially the feedback signals from our customers. And, regardless of what we try, things seem to get worse—we are no longer able to respond quickly to our changing competitive landscape, nor are we able

to provide stable, reliable service to our customers. As a result, we ultimately lose in the marketplace.

Time and time again, we learn that when IT fails, the entire organization fails. As Steven J. Spear noted in his book *The High-Velocity Edge*, whether the damages “unfold slowly like a wasting disease” or rapidly “like a fiery crash . . . the destruction can be just as complete.”⁵

Why Does This Downward Spiral Happen Everywhere?

For over a decade, the authors of this book have observed this destructive spiral occur in countless organizations of all types and sizes. We understand better than ever why this downward spiral occurs and why it requires DevOps principles to mitigate. First, as described earlier, every IT organization has two opposing goals, and second, every company is a technology company, whether they know it or not.

As Christopher Little, a software executive and one of the earliest chroniclers of DevOps, said, “Every company is a technology company, regardless of what business they think they’re in. A bank is just an IT company with a banking license.”^{6†} To convince ourselves that this is the case, consider that the vast majority of capital projects have some reliance on IT. As the saying goes, “It is virtually impossible to make any business decision that doesn’t result in at least one IT change.”

In the business and finance context, projects are critical because they serve as the primary mechanism for change inside organizations. Projects are typically what management needs to approve, budget for, and be held accountable for; therefore, they are the mechanism that achieves the goals and aspirations of the organization, whether it is to grow or even shrink.[§]

Projects are typically funded through capital spending (e.g., factories, equipment, and major projects, and expenditures are capitalized when payback is expected to take years), of which 50% is now technology related. This is even true in “low tech” industry verticals with the lowest

historical spending on technology, such as energy, metal, resource extraction, automotive, and construction.⁸ In other words, business leaders are far more reliant upon the effective management of IT in order to achieve their goals than they think.⁹

The Costs: Human and Economic

When people are trapped in this downward spiral for years, especially those who are downstream of Development, they often feel stuck in a system that preordains failure and leaves them powerless to change the outcomes. This powerlessness is often followed by burnout, with the associated feelings of fatigue, cynicism, and even hopelessness and despair.

Many psychologists assert that creating systems that cause feelings of powerlessness is one of the most damaging things we can do to fellow human beings—we deprive other people of their ability to control their own outcomes and even create a culture where people are afraid to do the right thing because of fear of punishment, failure, or jeopardizing their livelihood. This type of culture can create the conditions for learned helplessness, where people become unwilling or unable to act in a way that avoids the same problem in the future.

For our employees, it means long hours, working on weekends, and a decreased quality of life, not just for the employee, but for everyone who depends on them, including family and friends. It is not surprising that when this occurs, we lose our best people (except for those who feel like they can't leave because of a sense of duty or obligation).

In addition to the human suffering that comes with the current way of working, the opportunity cost of the value that we could be creating is staggering—the authors believe that we are missing out on approximately \$2.6 trillion of value creation per year, which is, at the time of this writing,

equivalent to the annual economic output of France, the sixth-largest economy in the world.

Consider the following calculation—both IDC and Gartner estimated that in 2011 approximately 5% of the worldwide gross domestic product (\$3.1 trillion) was spent on IT (hardware, services, and telecom).¹⁰ If we estimate that 50% of that \$3.1 trillion was spent on operating costs and maintaining existing systems, and that one-third of that 50% was spent on urgent and unplanned work or rework, approximately \$520 billion was wasted.

If adopting DevOps could enable us—through better management and increased operational excellence—to halve that waste and redeploy that human potential into something that’s five times the value (a modest proposal), we could create \$2.6 trillion of value per year.

The Ethics of DevOps: There Is a Better Way

In the previous sections, we described the problems and the negative consequences of the status quo due to the core, chronic conflict, from the inability to achieve organizational goals to the damage we inflict on fellow human beings. By solving these problems, DevOps astonishingly enables us to simultaneously improve organizational performance, achieve the goals of all the various functional technology roles (e.g., Development, QA, IT Operations, Infosec), and improve the human condition.

This exciting and rare combination may explain why DevOps has generated so much excitement and enthusiasm with so many and in such a short time, including technology leaders, engineers, and much of the software ecosystem we reside in.

Breaking the Downward Spiral with DevOps

Ideally, small teams of developers independently implement their features, validate their correctness in production-like environments, and have their

code deployed into production quickly, safely, and securely. Code deployments are routine and predictable. Instead of starting deployments at midnight on Friday and spending all weekend working to complete them, deployments occur throughout the business day when everyone is already in the office and without our customers even noticing—except when they see new features and bug fixes that delight them. And, by deploying code in the middle of the workday, IT Operations is working during normal business hours like everyone else for the first time in decades.

By creating fast feedback loops at every step of the process, everyone can immediately see the effects of their actions. Whenever changes are committed into version control, fast automated tests are run in production-like environments, giving continual assurance that the code and environments operate as designed and are always in a secure and deployable state.

Automated testing helps developers discover their mistakes quickly (usually within minutes), which enables faster fixes as well as genuine learning—learning that is impossible when mistakes are discovered six months later during integration testing, when memories and the link between cause and effect have long faded. Instead of accruing technical debt, problems are fixed as they are found, mobilizing the entire organization if needed because global goals outweigh local goals.

Pervasive production telemetry in our code and production environments ensures that problems are detected and corrected quickly, confirming that everything is working as intended and that customers are getting value from the software we create.

In this scenario, everyone feels productive—the architecture allows small teams to work safely and architecturally decoupled from the work of other teams who use self-service platforms that leverage the collective experience of Operations and Information Security. Instead of everyone waiting all the time, with large amounts of late, urgent rework, teams